

90. Une méthode qui en appelle une autre

Pourquoi chaque méthode a SES deux octets, et non deux partagés.

1. Ce qu'il faut comprendre

Dans `blessier()`, l'appel nu `vivant()` veut dire « **la mienne** » : l'objet en cours est passé tel quel, comme en C++.

Et voici pourquoi chaque méthode a **ses** deux octets plutôt que deux partagés par la `struct` : si `avancer()` appelait `autre.vivant()`, un emplacement commun serait écrasé, et `avancer()` reprendrait son travail sur le mauvais ennemi. Le partage aurait marché sur les exemples simples et échoué exactement là où c'est le plus dur à voir.

La règle du non-appel-à-soi-même vaut aussi ici : **une méthode ne peut pas s'appeler elle-même**, ni par un détour. Le compilateur nomme le cycle, méthode par méthode.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **687 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
Ennemi::vivant()	21 o	← Ennemi::blessier() — ta fonction
Ennemi::blessier()	39 o	← main() — ta fonction
main()	73 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	85 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	288 o	← CopierTuiles — 18 lettres sur 43
"VIVANT MORT"	11 o	← texte() — un texte gravé

Lettres gravées : AIMNORTV0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

nombre()	× 2	22 o sur place, dans main() + EcrireNombre (33 o)
texte()	× 1	11 o sur place, dans main() + EcrireTexte (10 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

```
Tes fonctions appelées :  
    Ennemi::vivant() × 1    15 o sur place, dans Ennemi::blesser()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : deux ennemis, un vivant (3 vies) et un mort (0 vie).  
// On blesse les deux : le vivant passe à 2, le mort reste à 0.  
//  
// Une méthode peut en appeler une autre : blesser() demande à vivant()  
// si l'ennemi a encore de la vie, avant d'en retirer.  
  
#include <texte>    // écrit un texte à l'écran  
#include <nombre>  // écrit un nombre en chiffres
```

2. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi {  
    uint8_t x, vie;  
  
    uint8_t vivant() { return vie > 0; }    // 1 s'il reste de la vie, sinon 0  
    void blesser()  { if (vivant()) vie--; } // perd une vie... seulement s'il est vivant  
};
```

3. UN TABLEAU — TROUPE

```
Ennemi troupe[2];    // 2 ennemis.
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

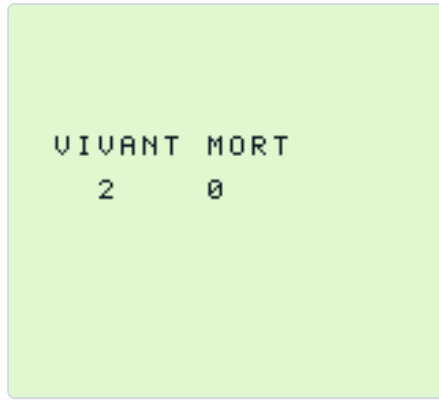
```
int main() {                // Le jeu commence ici.  
    troupe[0].vie = 3;      // le premier : vivant, 3 vies  
    troupe[1].vie = 0;      // le second : mort  
  
    troupe[0].blesser();    // 3 → 2  
    troupe[1].blesser();    // 0 → 0 : vivant() rend 0, rien ne se passe  
                            // (sans ce test, 0 - 1 donnerait 255 !)  
  
    texte(2, 6, "VIVANT MORT");  
    nombre(4, 8, troupe[0].vie, 1); // 2  
    nombre(9, 8, troupe[1].vie, 1); // 0  
  
    while (true) {          // Répète pour toujours :  
        image();            // attend l'image suivante.  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« 2 » et « 0 » — le mort n'est pas descendu à 255.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le vivant a perdu une vie
- ✓ le mort est resté à zéro
- ✓ il n'a PAS débordé à 255
- ✓ l'appel imbriqué n'a pas perdu l'objet

Ensuite : **90.1. Chapitre 14 — Les struct : ce qui va ensemble : ce qui vient** — chapitre 14 : Les struct : ce qui va ensemble. ·
gameboy3 — écrire une cartouche Game Boy en C++